



**Ministère de l'Enseignement Supérieur
Et de la Recherche Scientifique**



Rapport de Projet Personnel Professionnel 2

Filière : Génie Informatique

Réalisé par :

Cherni Mohamed Amine

Encadré par :

Encadrant académique : Mr Imed Ben Boukhatem

Encadrant professionnel : Mr Anis Zemni

Sujet : Développement d'une application web pour la gestion de méthodes de commerce extérieur

Année Universitaire : 2024/2025

Table Des Matières

INTRODUCTION GENERALE.....	1
CHAPITRE I : PRESENTATION DU CADRE DE PROJET.....	2
I. Introduction.....	2
II. Présentation de la « société d'accueil »	2
III. Problématique	3
IV. Solution proposée.....	4
V. Choix de méthodologie.....	4
VI. Conclusion.....	7
CHAPITRE II : ANALYSE DES BESOINS ET SPECIFICATIONS	
FONCTIONNELLES.....	8
I. Analyse des besoins.....	8
II. Spécifications fonctionnelles.....	8
III. Conclusion	10
CHAPITRE III : CONCEPTION DE PROJET.....	11
I. Introduction	11
II. Architecture Microservices.....	11
III. Modèle Mvc	13
IV. Diagrammes	14
V. Sécurité et Gestion des Accès.....	20
VI. Conclusion	21
CHAPITRE IV : REALISATION DE PROJET.....	22
I. Introduction	22
II. Environnement Technique.....	22
III. Présentation de l'interface utilisateur.....	34
IV. Tests avec Postman.....	42
V. Conclusion	47

Table Des Figures

Figure 1 Logo de la société « SMI »	2
Figure 2 Architecture Microservices.....	11
Figure 3 Modèle MVC.....	13
Figure 4 Diagramme des cas d'utilisation.....	15
Figure 5 Diagramme d'architecture Microservice.....	17
Figure 6 Diagramme de Séquence d'authentification	19
Figure 7 Diagramme de Séquence de la Création.....	19
Figure 8 Java	22
Figure 9 HTML	23
Figure 10 CSS	23
Figure 11 TypeScript	24
Figure 12 Spring Boot.....	24
Figure 13 IntelliJ IDEA	25
Figure 14 Angular	25
Figure 15 Bootstrap.....	26
Figure 16 Visuel Studio	26
Figure 17 SQL Server	27
Figure 18 Oracle.....	28
Figure 19 Keycloak.....	28
Figure 20 Service Discovery	30
Figure 21 GitHub	31
Figure 22 SQL Developer	32
Figure 23 Docker	33
Figure 24 Page de connexion	34
Figure 25 Page de Prise En Charge Accord BCT	34
Figure 26 Données Demmande(1)	35
Figure 27 Données Demmande(2)	35
Figure 28 Données Facture(1).....	36
Figure 29 Données Facture(2).....	36
Figure 30 Données Accord BCT/Assurances	37
Figure 31 Données NGP	37
Figure 32 Données Fichiers	38
Figure 33 Vérification.....	38
Figure 34 liste des pays.....	39
Figure 35 liste filtrée.....	39
Figure 36 Consultation d'un accord BCT	40
Figure 37 Liste des fichiers.....	41
Figure 38 Adaptation de la page avec la barre latérale	41
Figure 39 Test Authentification utilisateur	42
Figure 40 Test de Création de domiciliation.....	43
Figure 41 Test Création D'Accord BCT (1)	44
Figure 42 Test Création D'Accord BCT (2)	44
Figure 43 Test Liste des pays.....	45
Figure 44 Test Détails de l'accord BCT	46

Introduction générale

Le développement d'applications web est une discipline clé dans le domaine de l'informatique, combinant conception et implémentation d'outils interactifs pour répondre à des besoins spécifiques. Grâce à des technologies modernes telles que **Java Spring Boot**, **Angular**, et les bases de données relationnelles comme **Oracle**, il est possible de concevoir des solutions performantes adaptées aux processus métiers complexes.

Dans le monde des affaires d'aujourd'hui, les entreprises opèrent dans un environnement de plus en plus digitalisé, où l'automatisation et la centralisation des processus sont cruciales pour garantir efficacité et compétitivité. C'est particulièrement le cas dans le domaine bancaire et des opérations de commerce extérieur, où la rapidité et la précision des transactions sont essentielles.

Dans ce cadre, mon projet de stage professionnel au sein de **la Société Le Monde Informatique (SMI)** a pour objectif de moderniser une application existante en convertissant un système développé initialement avec Oracle Forms vers une architecture moderne basée sur **Java Spring Boot**. Cette initiative concerne spécifiquement le module de **Domiciliation des Titres de Commerce Extérieur**, un élément clé dans la gestion des opérations réglementaires de commerce extérieur.

Ce rapport décrit les différentes étapes du projet, de l'analyse des besoins à la mise en œuvre technique. Nous commencerons par présenter le contexte du projet, les objectifs, et les contraintes. Ensuite, nous décrirons les solutions techniques choisies, la conception et les étapes de réalisation, pour conclure avec une évaluation des résultats et perspectives d'amélioration.

Chapitre I : Présentation du cadre de projet

I. Introduction :

Dans ce chapitre, nous présentons le cadre global du projet en mettant en avant la société d'accueil et en définissant la problématique. Nous exposons également la solution proposée et analysons l'existant pour identifier les points à améliorer.

II. Présentation de la « société d'accueil » :

Mon stage de Projet professionnel s'inscrit au sein de la société SMI (Société Le Monde Informatique), Dont son logo est



Figure 1 Logo de la société « SMI »

Fondée par **Faycal Zemni en 1991**, la **Société Le Monde Informatique (SMI)** est spécialisée dans le développement de solutions bancaires, avec une expertise particulière dans les **opérations bancaires étrangères**. Forte de plus de 30 ans d'expérience, SMI s'est positionnée comme un acteur incontournable grâce à son système intégré appelé **IBANSYS**, qui couvre l'ensemble des besoins des banques en matière de gestion des opérations bancaires étrangères.[1]

Voici un aperçu des principales fonctionnalités offertes par **IBANSYS** :

- **Opérations réglementaires :**
 - ✓ Domiciliation des Titres de Commerce Extérieur
 - ✓ Apurement des Titres de Commerce Extérieur
 - ✓ Allocation pour voyages d'affaires (AVA)
- **Virements internationaux :**
 - ✓ Virements reçus
 - ✓ Virements commerciaux et financiers

- ✓ Pension de retraite
- ✓ Traitement des virements sur comptes en TND, devises ou dossiers de scolarité
- **Autres modes de règlement :**
 - ✓ Crédits documentaires
 - ✓ Remises documentaires (documents, chèques, effets)
 - ✓ Garanties internationales
 - ✓ Financements en devises
- **Gestion des messages SWIFT :**
 - ✓ Traitement automatisé de tous types de messages SWIFT (entrants et sortants)
 - ✓ Interprétation des messages SWIFT (XML ↔ MT)
 - ✓ Conformité avec les dernières mises à jour du standard SWIFT

Bien que le système **IBANSYS** repose actuellement sur des technologies robustes comme **Oracle Forms**, il présente certaines limitations, notamment en termes de flexibilité, de maintenance et d'intégration avec des solutions modernes. Cela justifie le besoin de modernisation.

III. Problématique :

L'application actuelle de gestion des Titres de Commerce Extérieur, bien qu'efficace dans ses fonctionnalités, est développée avec **Oracle Forms**, une technologie qui ne répond plus aux standards modernes de flexibilité, d'évolutivité, et d'expérience utilisateur. Les principales problématiques identifiées sont :

- **Complexité de maintenance** : Oracle Forms demande des compétences spécifiques, ce qui complique la maintenance et le recrutement de ressources techniques.
 - **Limitation en termes d'intégration** : L'intégration avec d'autres systèmes modernes ou web-based est difficile.
 - **Expérience utilisateur limitée** : L'interface utilisateur est vieillissante et peu intuitive.
 - **Accessibilité réduite** : L'application n'est pas optimisée pour une utilisation multiplateforme (mobile, tablette, web).
- Ces limitations impactent la productivité des utilisateurs et freinent l'évolution technologique de la société.

IV. Solution proposée :

Pour pallier ces limitations, le projet vise à **convertir l'application Oracle Forms en une architecture moderne basée sur Java Spring Boot** et des technologies front-end telles qu'Angular. La solution modernisée offre :

- **Modularité et maintenabilité** : Une architecture basée sur les **microservices**, facilitant les mises à jour et l'ajout de nouvelles fonctionnalités.
- **Interface utilisateur moderne** : Un **front-end** développé avec **Angular**, offrant une expérience utilisateur fluide et intuitive.
- **Accessibilité multiplateforme** : L'application sera accessible sur le web et adaptable.
- **Intégration facilitée** : Une architecture basée sur REST API, garantissant une intégration aisée avec d'autres systèmes bancaires et externes.

Le module de **Domiciliation des Titres de Commerce Extérieur**, premier à être modernisé, comprendra les fonctionnalités suivantes :

- Gestion complète des demandes de domiciliation (création, modification, suppression).
 - Consultation des statuts et suivi des titres domiciliés.
 - Automatisation des notifications et alertes pour les échéances.
 - Intégration avec les systèmes existants pour récupérer les données nécessaires (comme les détails des clients, RIB, etc.).
- Cette transition technologique répondra aux standards modernes tout en améliorant l'efficacité des utilisateurs et la valeur ajoutée de l'application.

V. Choix de Méthodologie :

1.Introduction

Le choix d'une méthodologie de gestion de projet est une étape cruciale dans la réalisation d'un projet informatique. Il détermine la manière dont les différentes phases du projet, de l'analyse à la mise en œuvre, sont structurées et exécutées. Plusieurs méthodologies ont été considérées pour sélectionner celle qui correspond le mieux aux objectifs et aux contraintes identifiées.

2. Comparaison des Méthodologies

A. Méthodologie Agile

La méthodologie Agile se caractérise par des itérations rapides et une collaboration étroite entre les membres de l'équipe. Elle est idéale pour les projets où les besoins peuvent évoluer au fil du temps.

Avantages :

- Flexibilité face aux changements.
- Livraison rapide des fonctionnalités.
- Forte implication des parties prenantes.

Inconvénients :

- Peut être difficile à gérer pour des équipes non expérimentées.
- Nécessite un engagement constant de la part des clients.

B. Cycle en V

Le cycle en V est une approche séquentielle où chaque phase du projet doit être terminée avant de passer à la suivante. Cette méthode est adaptée aux projets avec des besoins bien définis dès le départ.

Avantages :

- Documentation complète.
- Meilleure gestion des risques pour des projets bien structurés.

Inconvénients :

- Peu flexible face aux changements en cours de projet.
- Temps de retour long pour les tests.

C. Méthodologie Scrum

Scrum est une variante de l'Agile où le travail est organisé en sprints, des cycles courts permettant des itérations fréquentes.

Avantages :

- Favorise une communication efficace et un feedback rapide.
- Flexibilité dans la gestion des priorités.

Inconvénients :

- Dépend d'une forte discipline de l'équipe.
- Peut manquer de structure pour les grands projets complexes.

D. Méthodologie Kanban

Kanban est une méthode visuelle de gestion des flux de travail qui met l'accent sur l'optimisation des processus.

Avantages :

- Simplicité et adaptabilité.
- Amélioration continue des processus.

Inconvénients :

- Moins structuré pour les projets de grande envergure.
- Peut entraîner des retards si les priorités ne sont pas clairement définies

3.Critères d'Évaluation

Pour évaluer et choisir une méthodologie, les critères suivants ont été pris en compte :

- **Clarté des besoins initiaux** : La méthodologie doit permettre une bonne prise en compte des spécifications dès le début.
- **Complexité et taille du projet** : Une méthodologie adaptée doit être capable de gérer des projets avec de multiples dépendances.
- **Nécessité de flexibilité** : Le projet impliquant une modernisation technologique, une certaine souplesse était requise.
- **Implication des parties prenantes** : La collaboration étroite avec les utilisateurs finaux et les décideurs était essentielle.

4.Choix de la Méthodologie Adoptée

Après une analyse approfondie des méthodologies existantes, la méthodologie **Scrum** a été retenue pour ce projet. Cette approche répond parfaitement aux besoins du projet grâce aux éléments suivants :

- **Flexibilité** : Elle permet de s'adapter rapidement aux évolutions des besoins.
- **Itérations courtes** : Les sprints facilitent une livraison progressive et continue des fonctionnalités.
- **Collaboration active** : Scrum encourage une forte interaction entre l'équipe de développement et les parties prenantes.

➤ En conclusion, la méthodologie Scrum offre un équilibre optimal entre agilité et organisation structurée, ce qui est particulièrement pertinent dans le contexte de modernisation de l'application. Elle garantit que les objectifs du projet sont atteints tout en minimisant les risques liés aux imprévus.

VI. Conclusion :

Ce premier chapitre pose les bases du projet en expliquant les défis de l'existant et la vision apportée par la solution modernisée. Les chapitres suivants détailleront les spécifications techniques, la conception et la mise en œuvre du projet.

Chapitre II : Analyse des besoins et spécifications fonctionnelles

I. Analyse des besoins

L'analyse des besoins constitue une étape fondamentale pour assurer la réussite du projet. Elle consiste à identifier, documenter et prioriser les attentes des utilisateurs et des parties prenantes. Pour ce projet, plusieurs besoins ont été identifiés :

1. Besoins fonctionnels

- **Gestion des demandes de domiciliation** : création, modification et suppression des dossiers.
- **Consultation et suivi des dossiers domiciliés** : accès aux statuts, historique et détails.
- **Intégration des données** clients via des systèmes externes (comme les RIB et autres informations bancaires).

2. Besoins non fonctionnels

- **Performance** : Temps de réponse rapide même en cas de forte charge.
- **Sécurité** : Gestion robuste des accès et protection des données sensibles.
- **Accessibilité** : Compatibilité multiplateforme (web, mobile, tablette).
- **Extensibilité** : Prévoir l'ajout futur de nouvelles fonctionnalités.

II. Spécifications fonctionnelles

Les spécifications fonctionnelles traduisent les besoins identifiés en fonctionnalités précises et mesurables. Voici les principales :

1. Gestion des utilisateurs

- Création, modification et suppression des comptes utilisateurs.
- Gestion des rôles (administrateur, opérateur, etc.).

2. Gestion des demandes

- Formulaire interactif pour la création et la modification des dossiers.
- Consultation des dossiers avec filtres avancés (par date, statut, client, etc.).

3. Sécurité et authentification

- Intégration avec Keycloak pour l'authentification et l'autorisation.
- Protection des données via des protocoles sécurisés (HTTPS, chiffrement des mots de passe).

III. Conclusion

Ce chapitre a permis de détailler les besoins et spécifications du projet, garantissant ainsi une base solide pour la conception et la réalisation. Le prochain chapitre abordera la conception de l'application.

Chapitre III : Conception de projet

I. Introduction :

La conception d'un projet est une étape fondamentale pour garantir sa réussite et répondre aux besoins des utilisateurs. Ce chapitre détaille l'architecture et la conception de notre plateforme, qui repose sur une architecture moderne basée sur des microservices. Nous expliquerons l'organisation des différents services, leur rôle respectif, et la manière dont ils collaborent pour fournir une solution performante et évolutive. Nous détaillerons également les entités principales ainsi que les actions autorisées pour les utilisateurs.

II. Architecture Microservices :

1. Définition de l'architecture microservices

L'architecture microservices est une approche de conception logicielle où une application est décomposée en un ensemble de petits services indépendants. Chaque service est spécialisé dans une tâche ou un domaine métier précis et peut être déployé, mis à jour, ou redémarré individuellement sans affecter les autres services.

Les microservices communiquent généralement via des API REST ou des messages asynchrones. Cette architecture favorise l'évolutivité, la résilience et la facilité de maintenance des systèmes complexes.

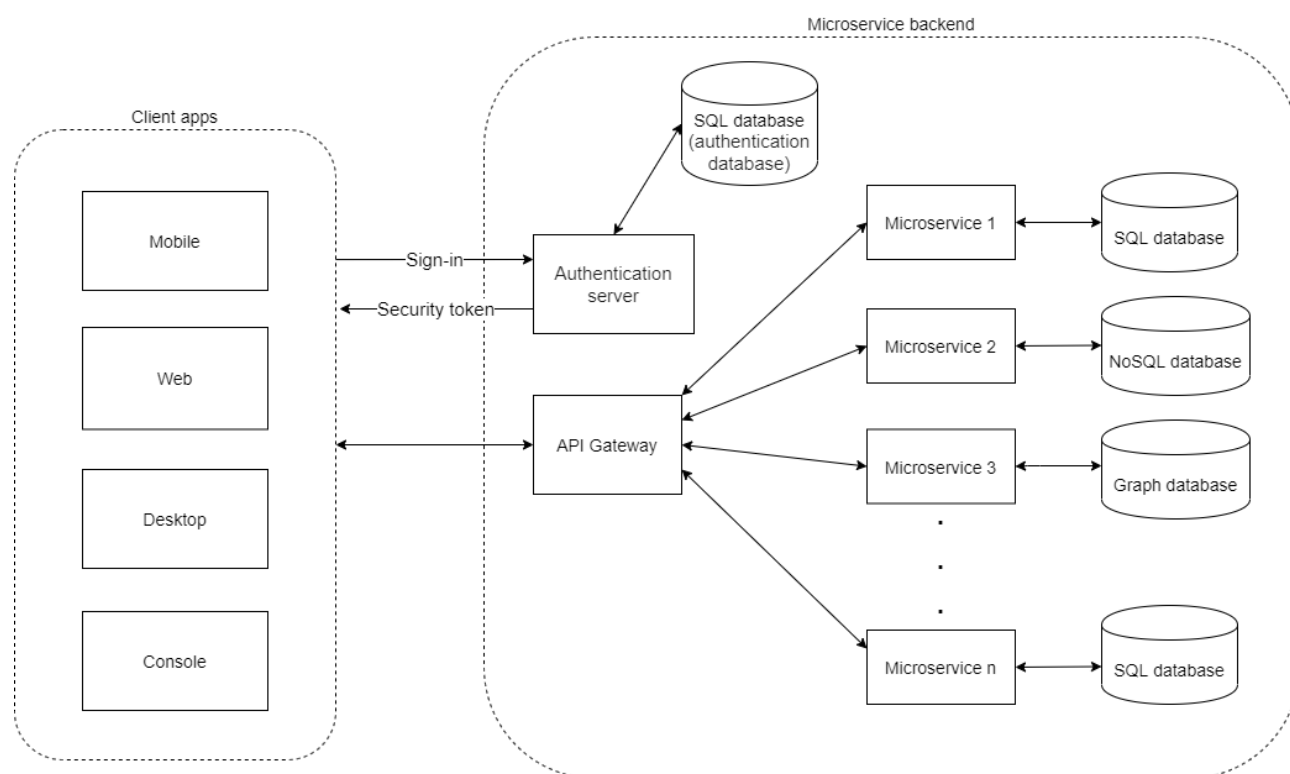


Figure 2 Architecture Microservices

2.Principaux Microservices du Projet

Pour répondre aux besoins d'évolutivité, de modularité et de maintenance, notre plateforme adopte une architecture basée sur des microservices. Chaque microservice est indépendant et spécialisé dans une tâche précise. Les principaux services développés sont les suivants :

a. Service d'authentification (Authentication)

- **Rôle** : Gère l'authentification et l'autorisation des utilisateurs via l'intégration de **Keycloak**.
- **Fonctionnalités** :
 - Authentification des utilisateurs via SSO (Single Sign-On).
 - Gestion des rôles et permissions via RBAC (Role-Based Access Control).

b. Service de configuration (Config-Server)

- **Rôle** : Centralise la gestion des configurations des autres microservices.
- **Fonctionnalités** :
 - Chargement des configurations depuis un dépôt Git.
 - Mise à jour dynamique des configurations des services via Spring Cloud Config.

c. Service de découverte (Discovery)

- **Rôle** : Sert de registre pour localiser les différents microservices.
- **Fonctionnalités** :
 - Découverte dynamique des services grâce à **Eureka**.
 - Monitoring des instances actives des microservices.

d. Passerelle d'API (Gateway)

- **Rôle** : Centralise et sécurise l'accès aux différents services.
- **Fonctionnalités** :
 - Redirection des requêtes utilisateur vers les services correspondants.
 - Mise en place de filtres pour la sécurité et la gestion des logs.

e. Service général (Gen)

- **Rôle** : Gère les fonctionnalités générales liées à la base de données nommée **Gen**.
- **Fonctionnalités** :
 - Gestion des entités, services, et contrôleurs associés à **Gen**.

f. Service de référence (RefProject)

- **Rôle** : Gère les fonctionnalités spécifiques liées à la base de données nommée **Ref**.
- **Fonctionnalités** :
 - Gestion des entités, services, et contrôleurs associés à **Ref**.

g. Service de domiciliation (Domi)

- **Rôle** : Assure la gestion des données et processus de domiciliation.
- **Fonctionnalités** :
 - Création, mise à jour et suppression des enregistrements de domiciliation.

3. Communication entre Microservices

Les microservices communiquent via des API REST exposées par chaque service. La passerelle (Gateway) joue un rôle central en facilitant cette communication et en garantissant une sécurité accrue grâce à l'intégration avec le service d'authentification.

III. Modèle MVC :

1. Définition du Modèle MVC

Le modèle **MVC (Model-View-Controller)** est une architecture logicielle qui divise une application en trois composants principaux :

- **Modèle (Model)** : Gère les données, la logique métier et les règles de l'application.
- **Vue (View)** : Est responsable de l'affichage des données à l'utilisateur.
- **Contrôleur (Controller)** : Reçoit les entrées de l'utilisateur, traite ces requêtes, et coordonne les interactions entre le modèle et la vue.

Cette séparation des responsabilités favorise la réutilisabilité du code et la maintenance.

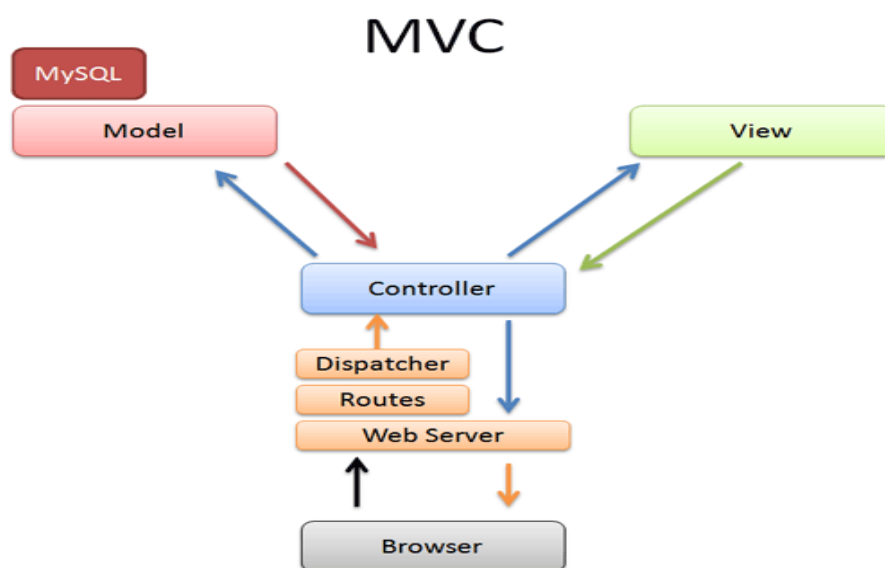


Figure 3 Modèle MVC

2. Application du Modèle MVC dans les Microservices

Chaque microservice suit une architecture interne basée sur le modèle MVC :

- **Modèle (Model)** : Définit les entités et interagit avec la base de données via des repositories Spring Data JPA.
- **Contrôleur (Controller)** : Gère les requêtes des utilisateurs, exécute les traitements nécessaires, et renvoie les réponses appropriées.
- **Service (Service)** : Contient la logique métier et orchestre les interactions entre le contrôleur et le modèle.

Cette structure garantit une organisation claire et facilite la maintenance de chaque service.

IV. Diagrammes :

1. Diagramme des cas d'utilisation

A. Définition

Un diagramme des cas d'utilisation illustre les interactions entre les utilisateurs (acteurs) et les différentes fonctionnalités (cas d'utilisation) d'un système. Il permet de modéliser les besoins fonctionnels en mettant en évidence les actions disponibles pour chaque utilisateur.

B. Types d'utilisateurs identifiés

Dans notre projet de domiciliation, deux principaux types d'utilisateurs ont été définis :

- **Utilisateur authentifié** : Toute personne accédant à l'application via un compte utilisateur, comme un opérateur ou un gestionnaire.
- **Administrateur** : Gère les utilisateurs, les paramètres, et les configurations globales du système.

C. Description des Cas d'Utilisation

C.1 Utilisateur authentifié

1. Créer une domiciliation :

- Ajoute de nouveaux enregistrements de domiciliation.
- Saisie des informations nécessaires (RIB, activités principales/secondaires, etc.).

2. Consulter les domiciliations :

- Affiche la liste des domiciliations existantes.
- Recherche des domiciliations par filtre (par date, statut, ou autre).

3. Mettre à jour une domiciliation :

- Modifie les données d'une domiciliation existante.

4. Supprimer une domiciliation :

- Supprime une domiciliation après vérification des permissions.

C.2. Administrateur

1. Gérer les utilisateurs :

- Ajouter, modifier ou supprimer des utilisateurs.
- Définir des rôles (opérateur, gestionnaire).

2. Configurer les paramètres système :

- Met à jour les configurations du système via le **Config-Server**.

3. Superviser les microservices :

- Contrôle la disponibilité des services via **Discovery**.

4. Consulter les journaux :

- Analyse des logs pour le suivi des opérations et la sécurité via **Gateway**.

D. Diagramme des cas d'utilisation

Le diagramme suivant représente les interactions possibles entre les utilisateurs et le système

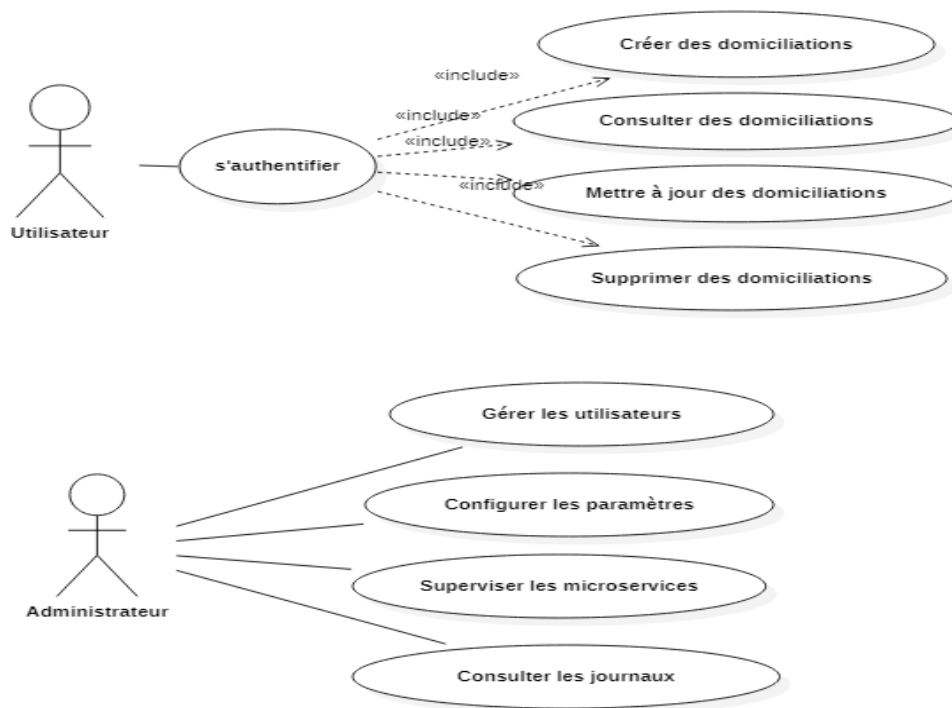


Figure 4 Diagramme des cas d'utilisation

Description du diagramme :

- Un acteur **Utilisateur authentifié** connecté à des cas d'utilisation : Créer, Consulter, Mettre à jour, Supprimer des domiciliations.
- Un acteur **Administrateur** connecté à des cas d'utilisation : Gérer les utilisateurs, Configurer les paramètres, Superviser les microservices, Consulter les journaux.

2. Diagramme d'architecture (Microservices)

A. Définition :

Un diagramme d'architecture des microservices est une représentation visuelle de la structure d'un système utilisant une architecture basée sur des microservices. Il illustre les principaux composants du système, les relations entre eux, et leurs interactions.

B. Description des Composants :

1. **Service d'authentification (Authentication)**
 - Gère l'authentification et l'autorisation des utilisateurs via Keycloak.
 - Fonctionnalités : Authentification, gestion des rôles et permissions.
2. **Service de configuration (Config-Server)**
 - Centralise la gestion des configurations des microservices.
 - Fonctionnalités : Chargement dynamique des configurations via Spring Cloud Config.
3. **Service de découverte (Discovery)**
 - Sert de registre pour localiser les différents microservices.
 - Fonctionnalités : Découverte et gestion des instances actives des microservices via Eureka.
4. **Passerelle d'API (Gateway)**
 - Centralise les requêtes et sécurise l'accès aux services.
 - Fonctionnalités : Redirection des requêtes, gestion des logs et de la sécurité.
5. **Service général (Gen)**
 - Gère les fonctionnalités générales de la base de données "Gen."
 - Fonctionnalités : Gestion des entités et des services associés.
6. **Service de référence (RefProject)**
 - Gère les fonctionnalités liées à la base de données "Ref."
 - Fonctionnalités : Gestion des références et des contrôleurs associés.
7. **Service de domiciliation (Domi)**
 - Assure la gestion des données et processus de domiciliation.
 - Fonctionnalités : Création, mise à jour, suppression des enregistrements.

C. Interactions :

- **Gateway → Authentification** : Contrôle des accès et redirection des utilisateurs.
- **Discovery → Tous les services** : Permet la découverte dynamique des services et la communication interservices.
- **Config-Server → Tous les services** : Fournit les configurations nécessaires à chaque service.
- **Gen, RefProject, Domi** : Interagissent via des API REST exposées.

D. Diagramme d'architecture :

Le diagramme ci-dessous illustre les microservices et leurs interactions :

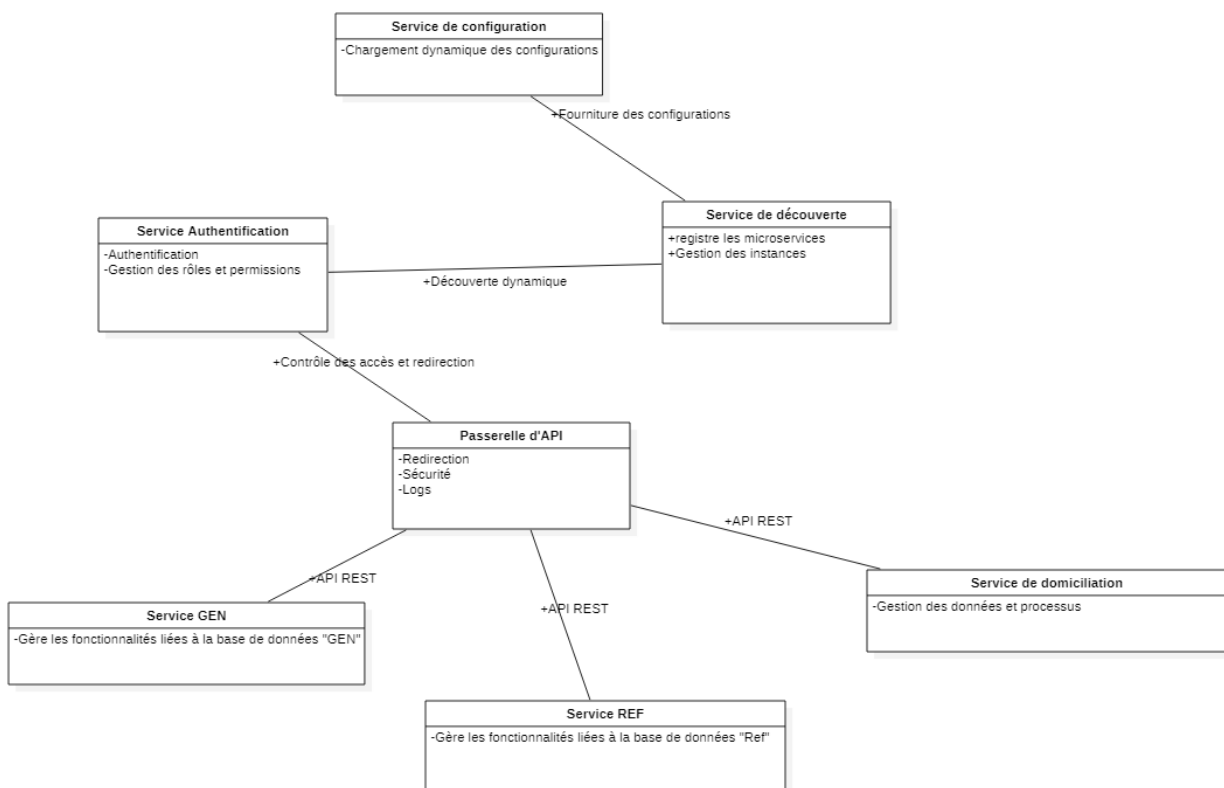


Figure 5 Diagramme d'architecture Microservice

3. Diagramme de séquence

A. Définition

Un diagramme de séquence est un diagramme UML utilisé pour modéliser l'ordre des interactions entre les différents composants d'un système. Il décrit comment les objets ou les entités

B. Scénarios Modélisés

Dans le cadre de notre projet de domiciliation, les diagrammes de séquence représentent les interactions essentielles entre les microservices. Les scénarios suivants ont été choisis pour mettre en avant les processus clés de l'application :

1. **Flux d'authentification via Keycloak** : Décrit le processus par lequel un utilisateur s'authentifie pour accéder au système.
2. **Création ou mise à jour d'une domiciliation** : Décrit les étapes nécessaires pour créer ou mettre à jour une domiciliation, incluant les interactions entre les services Domi et RefProject.

C. Explication des Scénarios

1. Flux d'authentification via Keycloak :

- L'utilisateur envoie ses informations de connexion à la passerelle d'API (Gateway).
- La Gateway relaie la requête au service d'authentification.
- Le service d'authentification interagit avec Keycloak pour valider les informations d'identification de l'utilisateur.
- Une fois authentifié, l'utilisateur reçoit un jeton d'accès pour accéder aux autres services.

2. Création ou mise à jour d'une domiciliation :

- L'utilisateur soumet une requête via l'interface utilisateur pour créer ou modifier une domiciliation.
- La Gateway transmet la requête au service Domi.
- Le service Domi interagit avec le service RefProject pour valider ou enrichir les données saisies.
- Une réponse est renvoyée à l'utilisateur via la Gateway.

D. Diagramme de Séquence

Les diagrammes suivants illustrent les interactions pour chaque scénario mentionné ci-dessus :

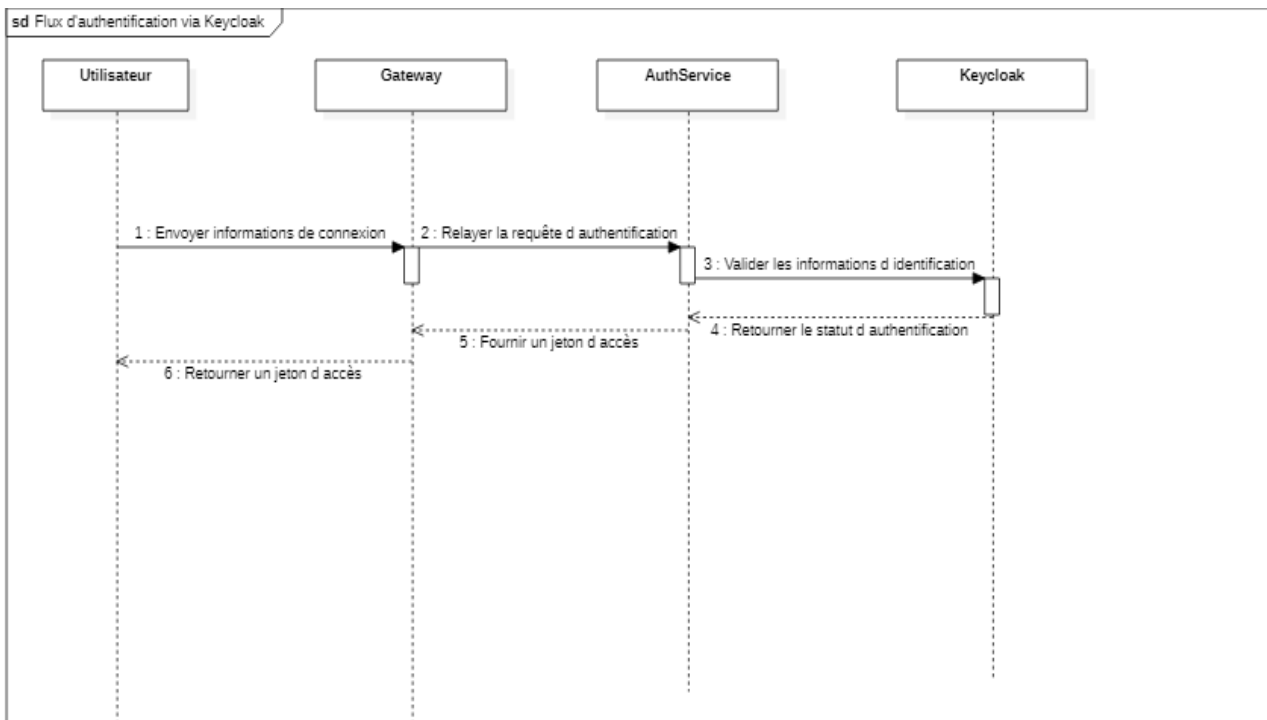


Figure 6 Diagramme de Séquence d'authentification

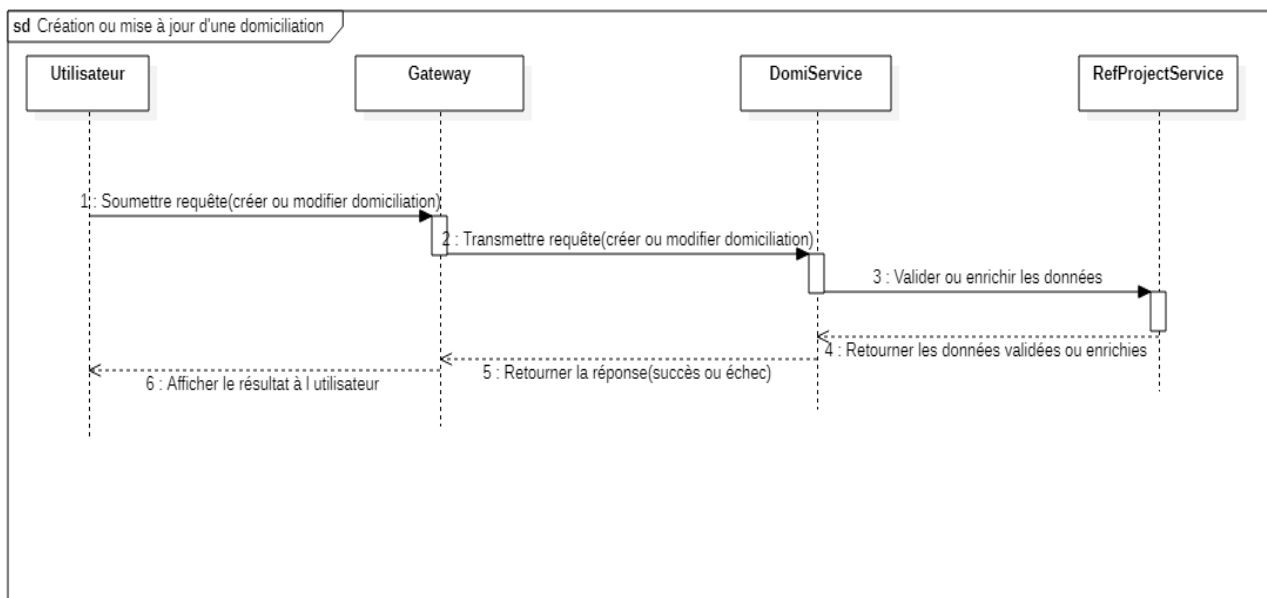


Figure 7 Diagramme de Séquence de la Création

V. Sécurité et Gestion des Accès

La sécurité du système repose sur les mécanismes suivants :

- **Keycloak** : Fournit une gestion centralisée des utilisateurs, des rôles, et des permissions.
- **Spring Security** : Renforce la protection des endpoints des microservices.
- **Chiffrement** : Garantit la confidentialité des données transmises entre services via HTTPS et des mots de passe hachés.

VI. Conclusions :

Dans ce chapitre, nous avons détaillé l'organisation et la conception de notre plateforme. L'approche microservices, combinée à une architecture basée sur MVC et une gestion centralisée de la sécurité, garantit une application robuste, évolutive, et maintenable.

Chapitre IV : Réalisation de projet

I. Introduction :

La réalisation de ce projet repose sur un environnement technique moderne, combinant outils, langages, frameworks, et bibliothèques pour garantir une application fonctionnelle, sécurisée et performante. Ce chapitre décrit les principaux éléments de cet environnement technique.

II. Environnement Technique

1. Langages de Programmation :

- **Java** : pour le développement **backend** avec **Spring Boot**.
- **HTML**, **CSS** et **JavaScript** : pour le développement **frontend**.

1.1 Java Langage :



Figure 8 Java

Java est un langage de programmation informatique de haut niveau et orienté objet, conçu pour être portable et robuste. Développé par Sun Microsystems (aujourd'hui Oracle Corporation), Java est largement utilisé pour les applications professionnelles grâce à sa fiabilité et à sa compatibilité multi-plateformes.

1.2 Langage de programmation HTML :



Figure 9 HTML

HTML signifie « HyperText Markup Language » qu'on peut traduire par « langage de balises pour l'hypertexte ». Il est utilisé pour créer la structure basique d'une page web et spécifier son contenu exemple titre, texte, des images, des liens et d'autres contenus afin de les afficher dans un navigateur web.

1.3 Langage de programmation CSS :



Figure 10 CSS

CSS est l'acronyme de « Cascading Style Sheets » ce qui signifie « feuille de style en cascade ». Le CSS correspond à un langage informatique permettant de mettre en forme des pages web.

1.4 Langage de programmation TypeScript :



Figure 11 TypeScript

TypeScript est un langage de programmation open-source développé par Microsoft. Il s'agit d'un sur-ensemble typé de JavaScript qui ajoute des fonctionnalités de typage statique au langage JavaScript standard. TypeScript est conçu pour améliorer la productivité des développeurs et la maintenabilité des grands projets JavaScript en détectant les erreurs de typage lors de la phase de développement.

2. Frameworks et Bibliothèques :

2.1 Spring Boot :



Figure 12 Spring Boot

- **Description** : Spring Boot est un framework Java qui simplifie et accélère le développement d'applications Java. Il offre une configuration par défaut pour démarrer rapidement et fournit des fonctionnalités puissantes pour le développement backend.
- **Rôle** : Spring Boot est utilisé pour créer l'infrastructure backend de l'application "Gestion Event", y compris la gestion des requêtes HTTP, la logique métier, la sécurité, l'accès aux données et bien plus encore.

2.2 IntelliJ IDEA :



Figure 13 IntelliJ IDEA

- **Description** : IntelliJ IDEA est un environnement de développement intégré (IDE) développé par JetBrains. Il offre des fonctionnalités avancées pour le développement Java, y compris la prise en charge de Spring Boot, la gestion de projet, le débogage, la refactorisation, et bien plus encore.
- **Rôle** : IntelliJ IDEA est l'IDE principal utilisé pour le développement de l'application "Gestion Event". Il fournit un environnement convivial et productif pour écrire, tester et déployer du code Java et Spring Boot.

2.3 Angular :



Figure 14 Angular

- **Description** : Angular est un framework JavaScript open-source développé par Google pour créer des applications web à une seule page (SPA). Il offre une structure robuste pour le développement d'interfaces utilisateur dynamiques et interactives.
- **Rôle** : Angular est utilisé pour créer les fonctionnalités frontend avancées de l'application "Gestion Event". Il permet de développer des composants réutilisables, d'établir des liens de données bidirectionnels, de gérer la navigation et d'interagir avec les API backend.

2.4 Bootstrap :



Figure 15 Bootstrap

- **Description :** Bootstrap est un framework CSS open-source qui facilite la conception de sites web réactifs et esthétiques. Il offre une collection de composants et de styles prêts à l'emploi pour créer des interfaces utilisateur attrayantes.
- **Rôle :** Bootstrap est utilisé pour la conception frontend de l'application "Gestion Event". Il fournit une mise en page réactive, des composants d'interface utilisateur pré-construits et des styles CSS personnalisables pour assurer une expérience utilisateur cohérente sur différentes plates-formes et appareils.

2.5 Editeur de texte visuel studio :



Figure 16 Visuel Studio

- **Description :** Visual Studio est un environnement de développement intégré (IDE) développé par Microsoft. Il prend en charge plusieurs langages de programmation, y compris JavaScript, et offre des fonctionnalités avancées pour le développement d'applications web et mobiles.

- **Rôle :** Visual Studio peut être utilisé pour le développement frontend et backend de l'application "Gestion Event" lorsqu'il est associé à des outils complémentaires tels que des extensions et des plugins pour la prise en charge de JavaScript, Angular et d'autres technologies web.

3. Base de Données :

3.1 SQL Server :



Figure 17 SQL Server

- **Description :**
SQL Server est un système de gestion de bases de données relationnelles (SGBDR) développé par Microsoft. Il est connu pour sa fiabilité, sa scalabilité, et ses outils intégrés qui facilitent la gestion des transactions et l'analyse des données.
- **Rôle dans le projet :**
 - Stockage des données opérationnelles utilisées par les microservices.
 - Gestion efficace des transactions liées aux opérations de l'application.
 - Assurer l'intégrité et la cohérence des données à travers les processus métiers.

3.2 Oracle Database :



Figure 18 Oracle

- **Description :**

Oracle Database est un SGBDR reconnu pour ses capacités à traiter de grandes quantités de données et à gérer des tâches complexes. Il offre des fonctionnalités avancées comme le partitionnement, le clustering, et la prise en charge des bases de données distribuées.

- **Rôle dans le projet :**

- Utilisé pour des fonctionnalités spécifiques nécessitant des performances optimales, notamment pour le traitement de données complexes.
- Support des fonctionnalités critiques et des opérations analytiques.
- Intégration avec les microservices via des requêtes SQL optimisées

4. Sécurité :

4.1 Keycloak



Figure 19 Keycloak

- **Description :**

Keycloak est une solution open-source de gestion des identités et des accès (IAM) développée par Red Hat. Elle permet d'ajouter des fonctionnalités d'authentification, d'autorisation, et de gestion des utilisateurs de manière centralisée dans une application ou un ensemble de microservices.

- **Rôle dans le projet :**

Dans notre projet, **Keycloak** a été intégré en tant que serveur d'authentification et d'autorisation. Il permet de :

- Gérer les utilisateurs, rôles et permissions.
- Implémenter l'authentification via OAuth2.0 et OpenID Connect (OIDC).
- Protéger les endpoints des microservices en validant les jetons d'accès (access tokens) générés par Keycloak.
- Faciliter la gestion des sessions utilisateurs et fournir une solution unifiée pour la sécurité.

- **Fonctionnalités mises en œuvre :**

- Configuration des clients pour chaque microservice dans Keycloak.
- Utilisation de **Realms** pour séparer les contextes d'application.
- Authentification basée sur JWT (JSON Web Token) pour sécuriser les échanges entre les services.
- Intégration avec les outils de backend (Spring Boot) via l'adaptateur Keycloak.

4.2 Service Discovery :

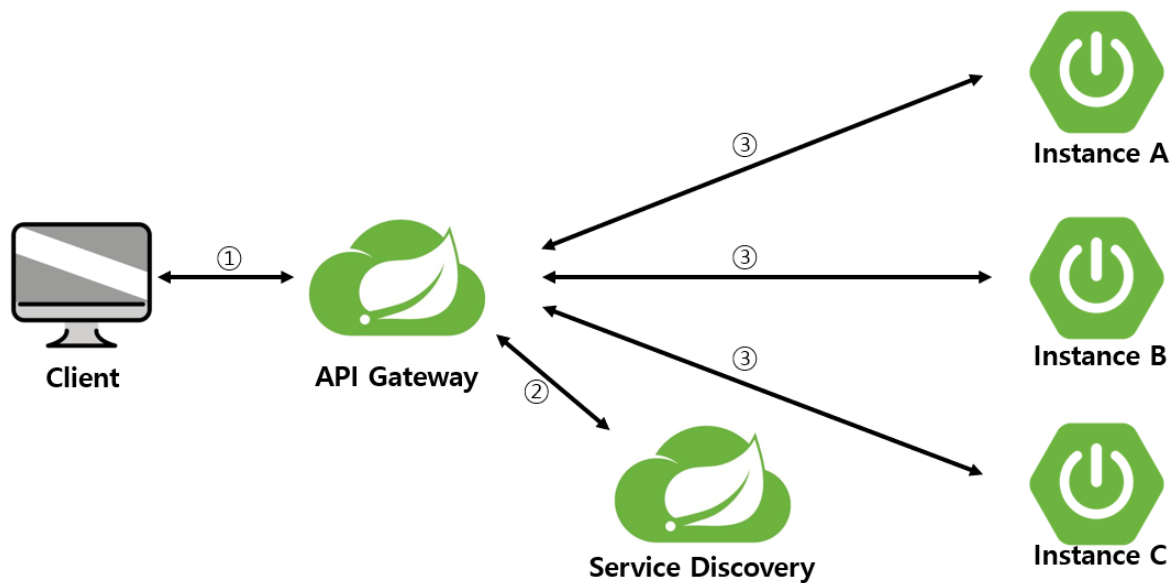


Figure 20 Service Discovery

- **Description :**

Le **Service Discovery** est une technique permettant de détecter et de gérer dynamiquement les instances de microservices au sein d'un système distribué. Ce mécanisme est essentiel pour gérer la communication sécurisée entre les microservices, surtout dans une architecture basée sur des conteneurs (par exemple, via Docker).

- **Rôle dans le projet :**

- Identifier automatiquement les microservices et leurs instances à travers un registre central.
- Assurer la connectivité et la disponibilité des services de manière dynamique, même en cas de montée en charge ou d'échec d'une instance.
- Faciliter l'intégration avec Keycloak pour garantir que les appels interservices soient sécurisés et autorisés.

- **Technologie utilisée :**

- **Eureka Server** a été déployé pour gérer le registre des microservices.
- Chaque microservice s'enregistre automatiquement dans le registre et est protégé par des règles de sécurité définies dans Keycloak.

5. Outils de Développement :

5.1 IDE :

- **IDE** : Environnement de développement intégré tel que **IntelliJ IDEA** est utilisé pour écrire, tester et déboguer le code.

5.2 GITHUB :



Figure 21 GitHub

- **Description :**

GitHub est une plateforme basée sur Git qui permet d'héberger des dépôts de code source en ligne, tout en offrant des outils collaboratifs comme les pull requests, les issues, et la gestion des projets.

- **Rôle dans le projet :**

- Hébergement sécurisé du code source du projet.
- Gestion des versions et collaboration en équipe.
- Utilisation de fonctionnalités telles que **pull requests** pour réviser et intégrer les modifications.
- Documentation et suivi des tâches via les issues.

5.2 SQL Developer :



Figure 22 SQL Developer

- **Description :**

Oracle SQL Developer est un environnement de développement intégré (IDE) utilisé pour gérer et interagir avec les bases de données Oracle. Cet outil simplifie les opérations sur les bases de données telles que la création de tables, l'exécution de requêtes SQL, et la gestion des schémas.

- **Rôle dans le projet :**

- Gestion et développement de la base de données SQL Server et Oracle.
- Conception et modification des tables, des relations, et des contraintes.
- Écriture, test et optimisation des requêtes SQL pour accéder aux données de manière efficace.
- Analyse des performances des requêtes et ajustements si nécessaire.

- Avec l'intégration de **Git**, **GitHub**, et **SQL Developer**, l'équipe a pu gérer efficacement le code source et interagir de manière optimale avec la base de données pour garantir un développement fluide et performant.

6. Compatibilité :

Assurer la compatibilité avec les navigateurs web modernes tels que Chrome, Firefox, Safari et Edge.

8. Hébergement :

L'application est hébergée via Docker, ce qui garantit une mise en œuvre homogène entre les environnements de développement, de test et de production.

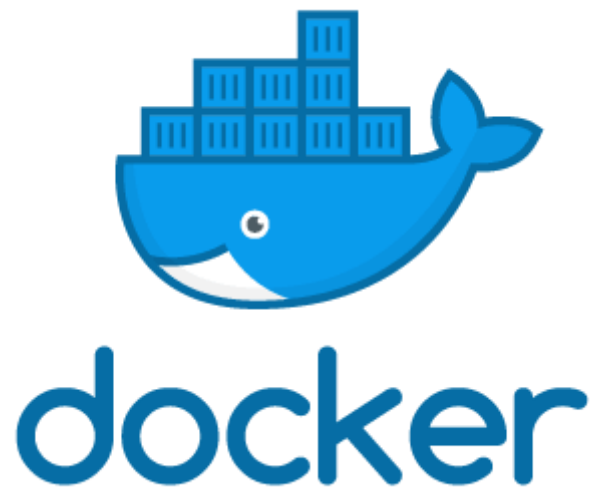


Figure 23 Docker

III. Présentation de l'interface utilisateur :

Cette section illustre l'interface utilisateur de l'application développée, mettant en avant les principales fonctionnalités et la disposition des éléments. Les captures d'écran suivantes montrent les pages et composants clés :

1. **Page de connexion** : Présente un formulaire d'authentification pour les utilisateurs, permettant une connexion sécurisée à l'application.

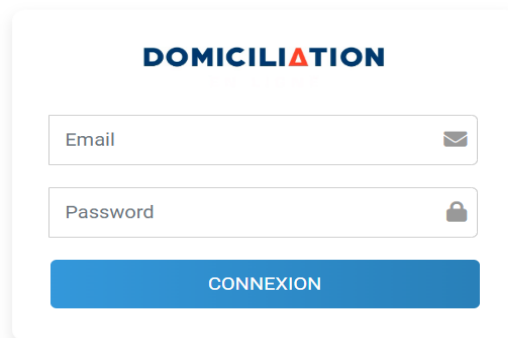


Figure 24 est une capture d'écran de la page de connexion de l'application. Elle présente un formulaire centré avec le logo 'DOMICILIATION' en haut. Le formulaire contient deux champs de saisie : 'Email' avec une icône d'adresse e-mail à droite, et 'Password' avec une icône de cadenas à droite. En dessous de ces champs se trouve un bouton bleu rectangulaire avec l'inscription 'CONNEXION' en lettres capitales.

Figure 24 Page de connexion

2. **Page de Prise En Charge Accord BCT** : Inclut un formulaire de création pour les utilisateurs, facilitant la gestion des accords BCT.

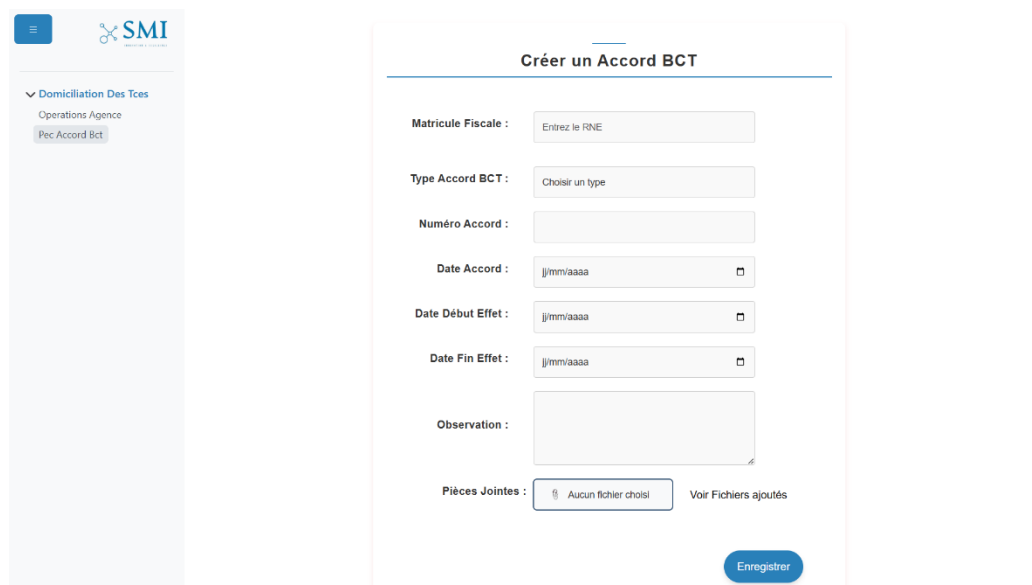


Figure 25 est une capture d'écran de la page 'Créer un Accord BCT'. À gauche, une barre latérale blanche contient le logo 'SMI' et un menu déroulant 'Domiciliation Des Tces' avec les options 'Operations Agence' et 'Pec Accord Bct'. Le formulaire principal, intitulé 'Créer un Accord BCT', est blanc et contient plusieurs champs : 'Matricule Fiscale' (avec le placeholder 'Entrez le RNE'), 'Type Accord BCT' (avec le placeholder 'Choisir un type'), 'Numéro Accord', 'Date Accord', 'Date Début Effet', et 'Date Fin Effet' (tous avec des calendriers iconiques). Il y a également un champ 'Observation' et une section 'Pièces Jointes' avec un bouton 'Aucun fichier choisi' et un lien 'Voir Fichiers ajoutés'. Un bouton bleu 'Enregistrer' est situé en bas à droite.

Figure 25 Page de Prise En Charge Accord BCT

3. **Page d'Opération Agence** : Permet aux utilisateurs de gérer les opérations au sein de l'agence, avec des fonctionnalités adaptées à leurs besoins.

a. **Données Demande**

Remplir tous les champs du formulaire pour passer à l'étape suivante !

1 2 3 4 5 6

Données Demande

Nature de la demande
Selectionnez le type de demande

Code du titre
Selectionnez le code titre

Code Agence
Entrez le code de l'agence 🔍

Matricule Fiscale
Entrez le RNE

Compte RIB

Totallement exportatrice :
☐ Oui ☒ Non

Figure 26 Données Demande(1)

Code du titre
Selectionnez le code titre

Code Agence
Entrez le code de l'agence 🔍

Matricule Fiscale
Entrez le RNE

Compte RIB

Totallement exportatrice :
☐ Oui ☒ Non

Type d'activité :
☒ Primaire ☐ Secondaire

Activité principale :
Code 🔍 Libellé de l'activité

Suivant

Figure 27 Données Demande(2)

b. Données Facture

SMI

Domiciliation Des Tces

Operations Agence

Pec Accord Bct

Remplir tous les champs du formulaire pour passer à l'étape suivante !

1

2

3

4

5

6

Données Facture

Número Contrat Commerciale

Número facture

Date Contrat Commerciale

jj/mm/aaaa

Nombre de colis

Nombre de colis

Devise

Code

Libellé de la devise

Mode de livraison

Code

Libellé du mode de livraison

Acompte/Avance

0 - 100%

%

Montant PTFN (DVS)

Montant PTF

Montant PTFN (TND)

Montant PTF

Montant FOB (DVS)

Montant FOB

Montant FOB (TND)

Montant FOB

Figure 28 Données Facture(1)

SMI

Domiciliation Des Tces

Operations Agence

Pec Accord Bct

Nom fournisseur

Adresse fournisseur

Nom fournisseur

Adresse fournis

Pays d'origine

Code

Libellé du pays

Pays d'achat

Code

Libellé du pays

Pays de provenance

788

TUNISIE

Pays de destination

788

TUNISIE

Mode de règlement

Code

Libellé du mode de règlement

Délai de règlement

Code

Libellé du délai de règlement

Régime statistique

Code

Libellé du régime statistique

Précédent

Suivant

Figure 29 Données Facture(2)

c. Données Accord BCT/Assurance

Remplir tous les champs du formulaire pour passer à l'étape suivante !

1 2 3 4 5 6

Accord BCT/Assurance

Accord BCT

Type d'accord BCT
Sélectionnez le type d'accord

Numéro d'accord BCT
Sélectionnez le numéro d'accord BCT

Date d'accord BCT

Observation

[Consulter](#)

Accord Assurance

☒ Assurance ☐ Garantie Bancaire

Numéro Assurance
Numéro Assurance

Date Assurance
jj/mm/aaaa

[Précédent](#) [Suivant](#)

Figure 30 Données Accord BCT/Assurances

d. Données NGP

Remplir tous les champs du formulaire pour passer à l'étape suivante !

1 2 3 4 5 6

Articles NGP

Rébrique:
1

NGP	Montant NGP	Pays d'origine	Unité	Quantité NGP
	<i>i</i> Montant NGP			

[Précédent](#) [Suivant](#)

Figure 31 Données NGP

e. Données Fichiers

The screenshot shows the 'Données Fichiers' step of a 6-step process. The left sidebar contains the SMI logo and a menu with 'Domiciliation Des Tces', 'Operations Agence', and 'Pec Accord Bct'. The main content area has a progress bar with steps 1-6, where step 5 is active. Below the progress bar, it says 'Remplir tous les champs du formulaire pour passer à l'étape suivante !'. A 'Charger un fichier' button is present. Under 'Sélectionner un fichier :', there is a file selection box showing 'Aucun fichier choisi' and a 'Voir Fichiers ajoutés' link. At the bottom are 'Précédent' and 'Suivant' buttons.

Figure 32 Données Fichiers

f. Vérification

The screenshot shows the 'Vérification' step of the 6-step process. The left sidebar is identical to the previous step. The main content area has a progress bar with steps 1-6, where step 6 is active. Below the progress bar, it says 'Remplir tous les champs du formulaire pour passer à l'étape suivante !'. The 'Vérification' section contains a list of items to verify: 'Données Demande', 'Données Facture', 'Accord BCT/Assurance', 'Données NGP', and 'Données FICHIER'. Each item has a document icon and a right-pointing arrow. At the bottom are 'Précédent' and 'Soumettre' buttons.

Figure 33 Vérification

4. Exemples de fonctionnalités sur la page:

a-liste des pays :

- Permet aux utilisateurs de sélectionner un pays d'origine à partir d'une liste.
- **Fonctionnalités :**
- Un champ de saisie pour le code du pays, avec un bouton de recherche qui ouvre une modal pour sélectionner un pays.
- La modal contient une barre de recherche pour filtrer les pays, ainsi qu'un tableau affichant les codes et libellés des pays disponibles.
- Lorsqu'un pays est sélectionné, son libellé est affiché sous le champ de saisie.

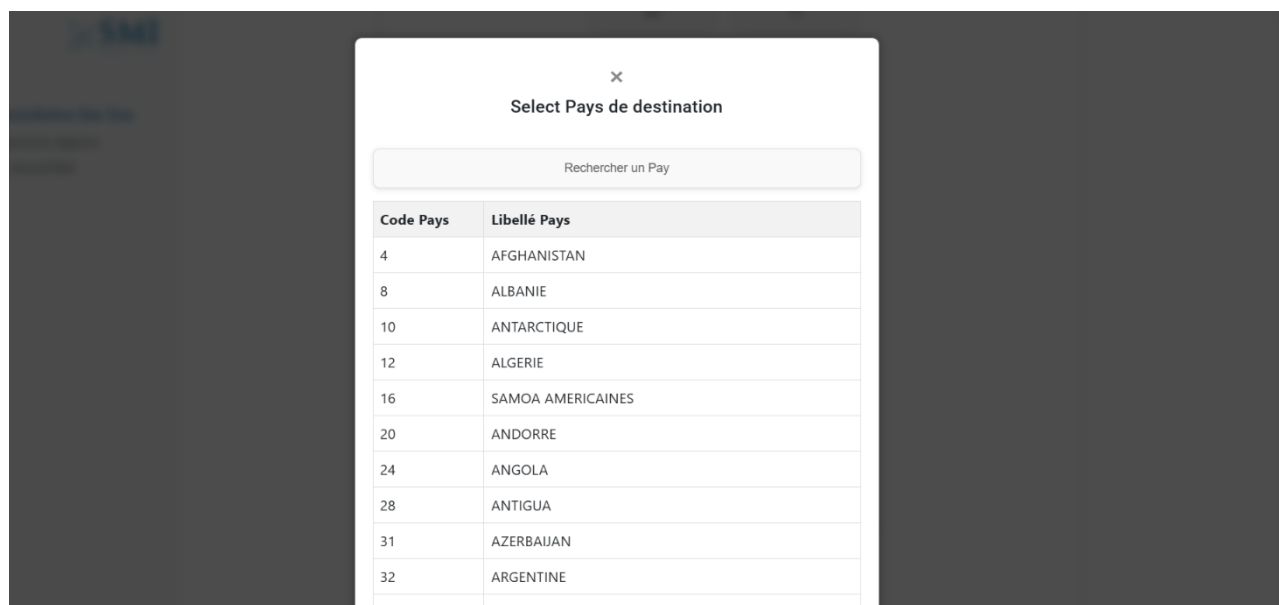


Figure 34 liste des pays

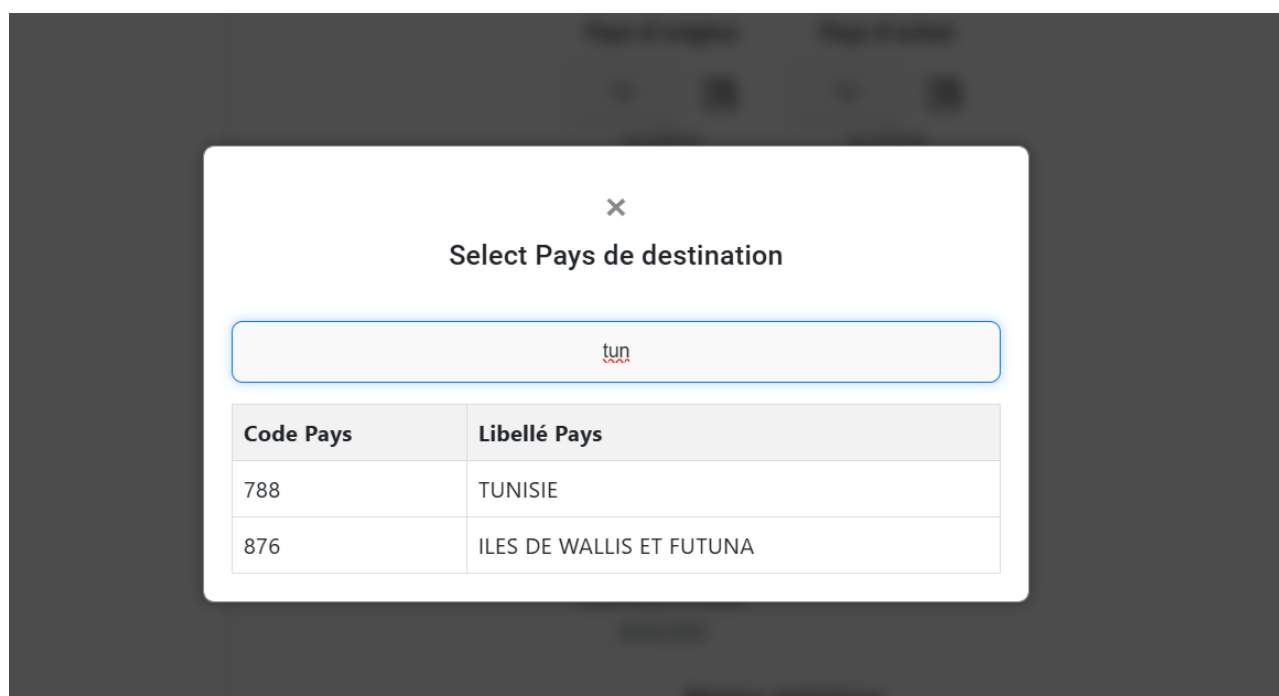


Figure 35 liste filtrée

b- Consultation d'un accord BCT

- Permet aux utilisateurs de consulter les détails d'un accord BCT spécifique en cliquant sur un lien de consultation.
- **Fonctionnalités :**
- Un bouton "Consulter" qui ouvre une modal affichant les détails de l'accord BCT sélectionné, y compris des informations telles que le type de pièce client, les dates d'application,

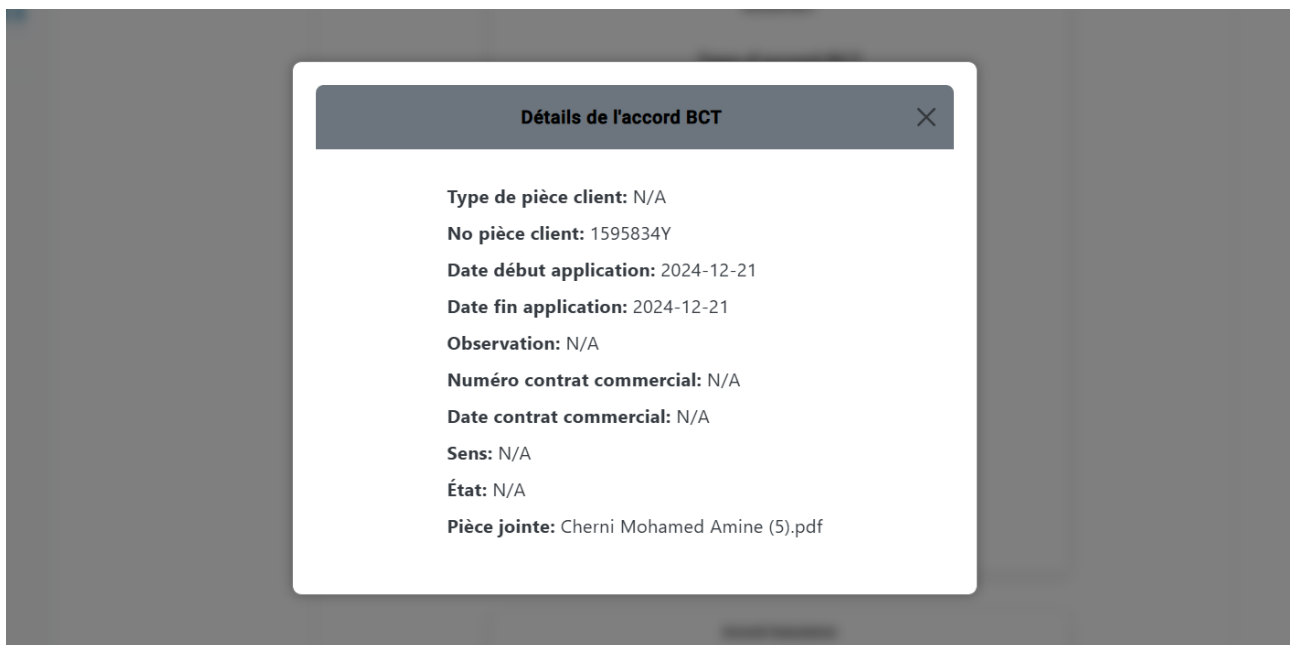


Figure 36 Consultation d'un accord BCT

c- Liste des fichiers

- Permet aux utilisateurs de télécharger des fichiers et de visualiser la liste des fichiers ajoutés.
- **Fonctionnalités :**
- Un champ de téléchargement de fichiers qui permet de sélectionner plusieurs fichiers.
- Un bouton pour afficher la liste des fichiers ajoutés, qui ouvre une modal contenant les noms des fichiers et des options pour ouvrir ou supprimer chaque fichier.

Numéro Accord :

Date Accord :

Date Début Effet :

Date Fin Effet :

Fichiers ajoutés

	NOM DU FICHIER	ACTION
1	Cherni Mohamed Amine (5).pdf	
2	DIAGRAMME PP2.mdj	

Enregistrer

Figure 37 Liste des fichiers

d. Adaptation de la page avec la barre latérale :

- Lorsque la barre latérale est fermée, le contenu de la page se recentre pour occuper tout l'espace disponible. Lorsque la barre latérale est rouverte, la page revient à son état initial, avec le contenu affiché à côté de la barre latérale.

Créer un Accord BCT

Matricule Fiscale :

Type Accord BCT :

Numéro Accord :

Date Accord :

Date Début Effet :

Date Fin Effet :

Observation :

Pièces Jointes : [Voir Fichiers ajoutés](#)

Enregistrer

Figure 38 Adaptation de la page avec la barre latérale

2. Création de domiciliation : Envoi d'une requête POST avec les détails de la domiciliation.

- **URL** : <http://localhost:8080/api/domiciliations>
- **Body** : Informations sur la domiciliation (client, montant, date).
- **Réponse attendue** : ID de la domiciliation et statut 200 Created.

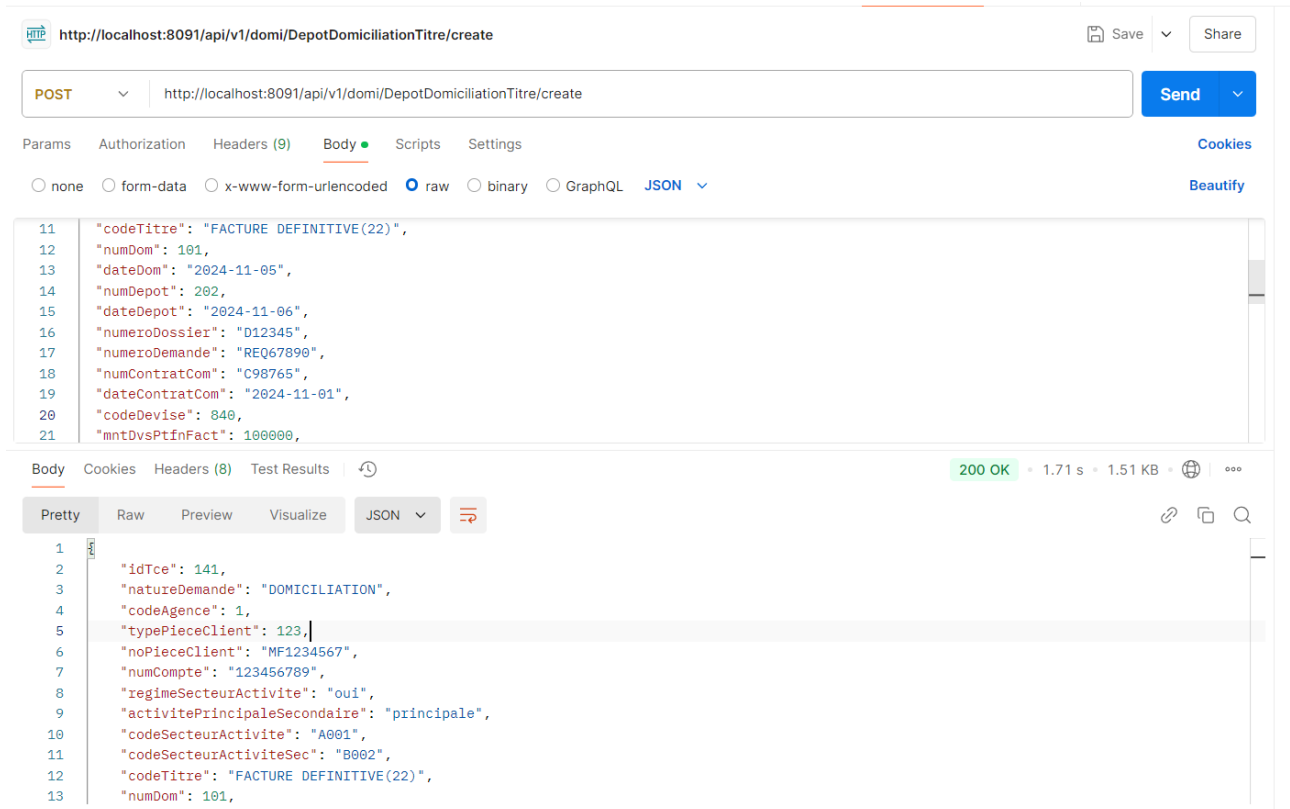


Figure 40 Test de Création de domiciliation

3. Création D'Accord BCT :

1^{er} cas : Envoi d'une requête POST avec les détails de l'accord.

- **URL** : <http://localhost:8092/api/v1/ref/AccordBct/create>
- **Réponse attendue** : Accord cree avec succes avec statu 200 OK

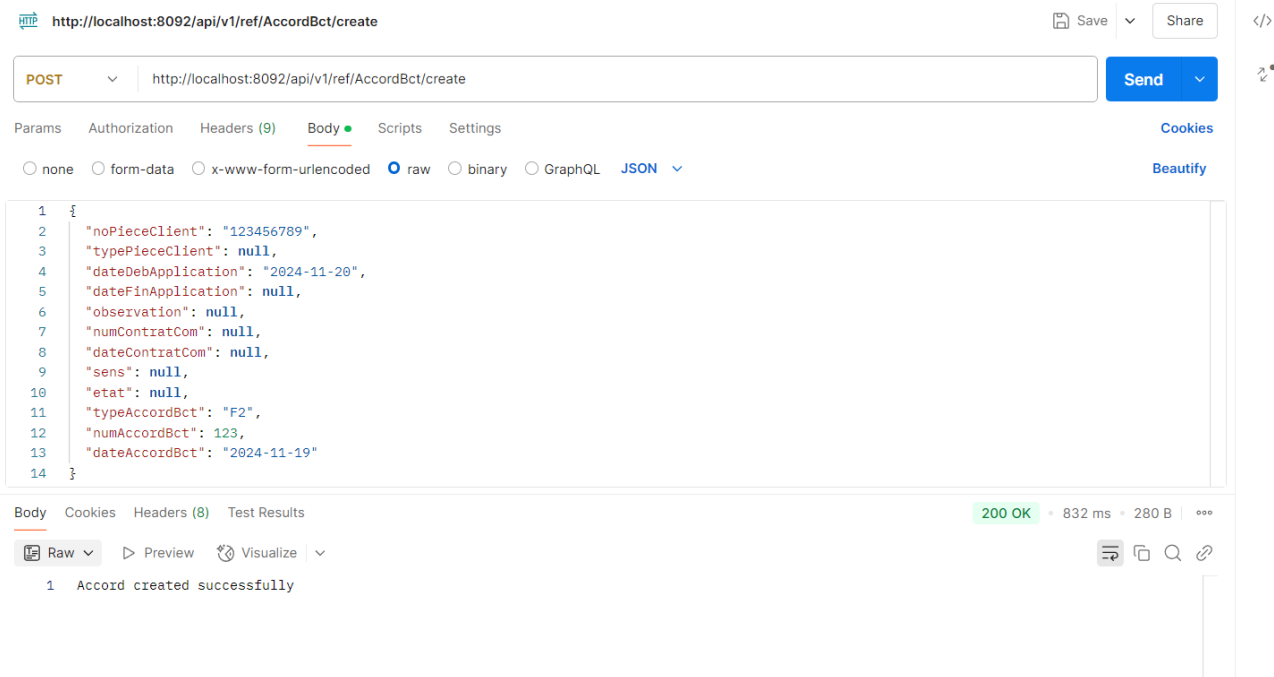


Figure 41 Test Création D'Accord BCT (1)

2^{ème} cas : Envoi d'une requête POST avec les détails de l'accord + fichier(s).

- URL : <http://localhost:8093/documents/uploadAndSave>
- Réponse attendue : Accord cree avec succes avec statu 200 OK

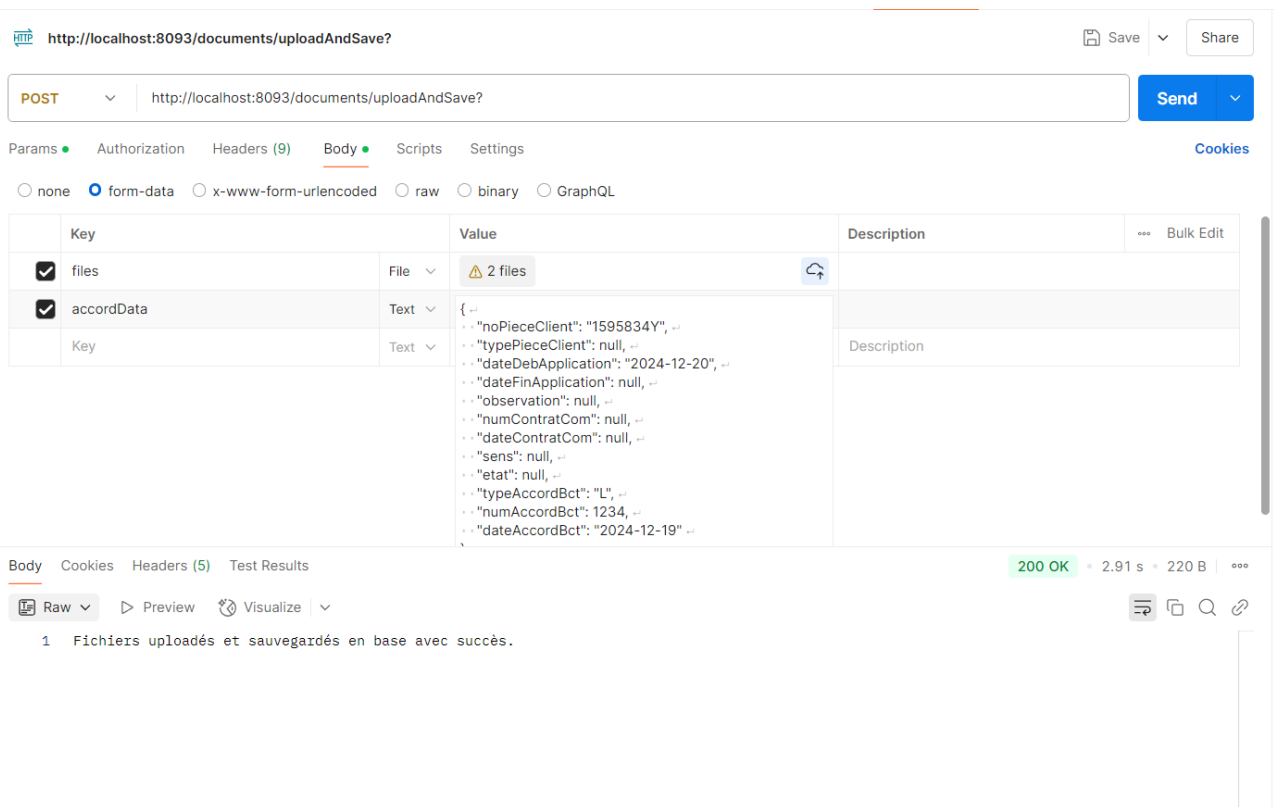


Figure 42 Test Création D'Accord BCT (2)

4. **Liste des pays** : Envoi d'une requête GET pour obtenir la liste complète des pays avec tous leurs détails disponibles dans la base de données.

- **URL** : `http://localhost:8092/api/v1/ref/Pays/all`
- **Réponse attendue** : Liste des pays avec leurs détails et statut 200 OK.

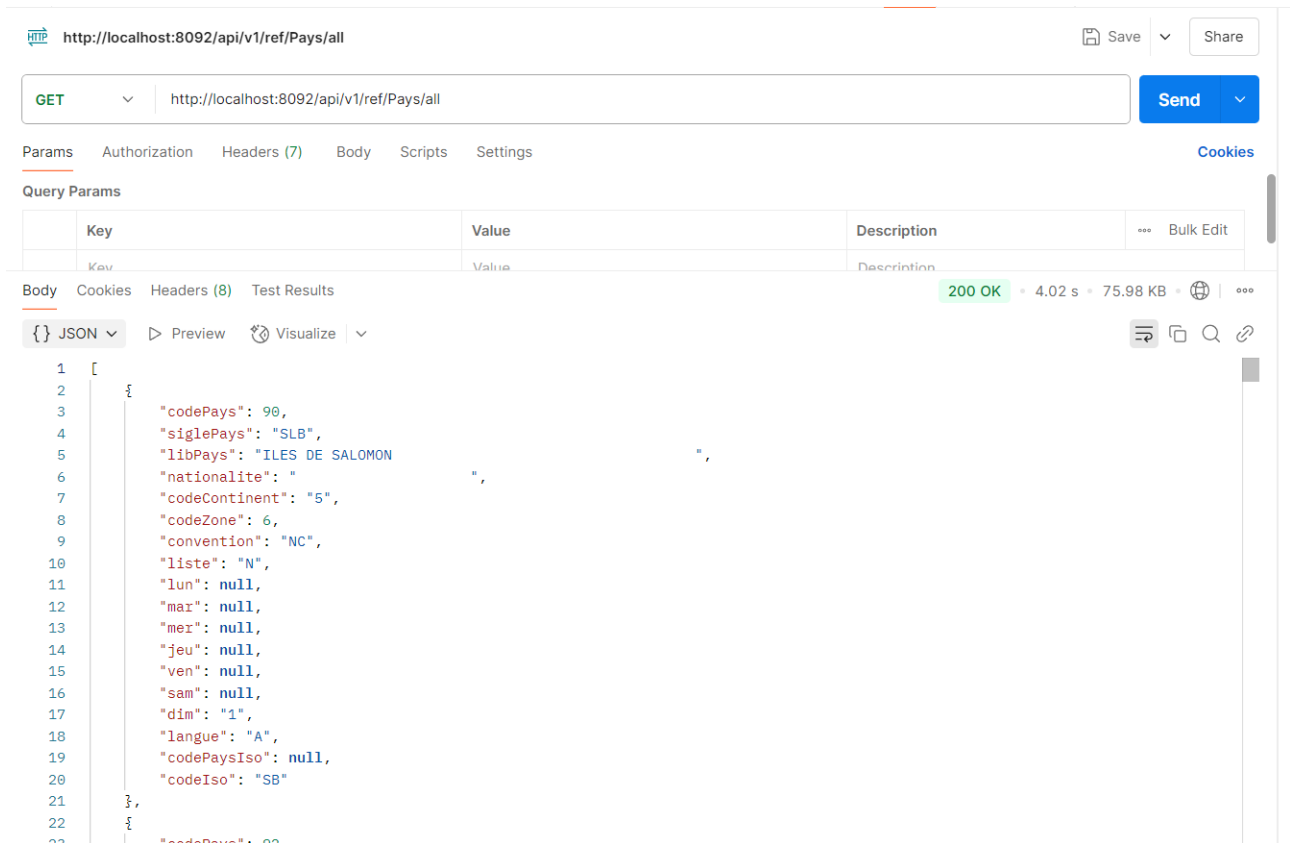


Figure 43 Test Liste des pays

5. **Détails de l'accord BCT** : Envoi d'une requête GET pour récupérer les détails d'un accord BCT spécifique à partir de son type, de son numéro, et de sa date d'accord.

- **URL** : `http://localhost:8092/api/v1/ref/Accord/detailaccordbct/{typeAccordBct}/{numAccordBct}/{dateAccordBct}`

- **Paramètres :**
 1. **typeAccordBct** : Le type de l'accord BCT.
 2. **numAccordBct** : Le numéro unique de l'accord BCT.
 3. **dateAccordBct** : La date de l'accord au format YYYY-MM-DD.
- **Réponse attendue** : 200 OK si l'accord BCT est trouvé.

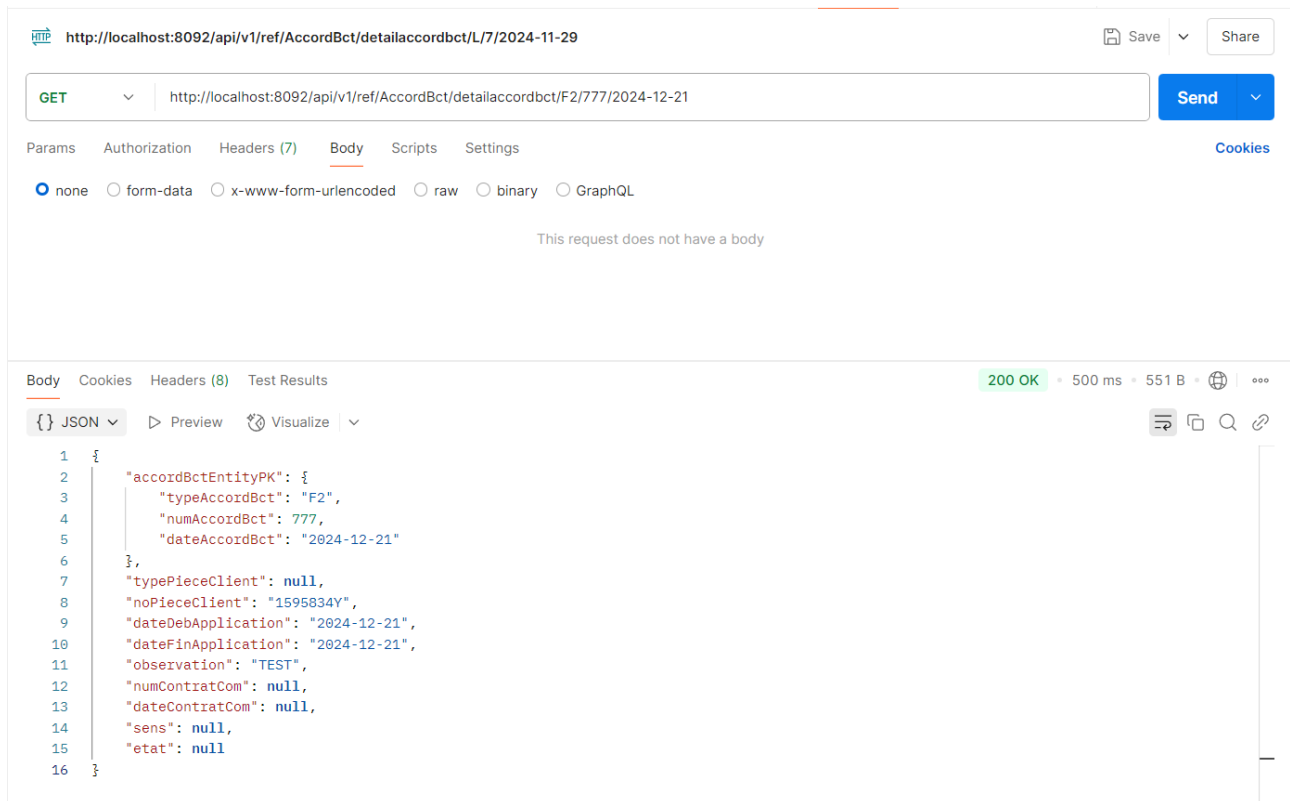


Figure 44 Test Détails de l'accord BCT

3. Analyse des Résultats

Chaque scénario a été testé avec succès, confirmant la conformité des API aux exigences. Les tests ont permis d'identifier quelques points d'amélioration, comme le traitement des erreurs et les validations côté serveur.

V. Conclusion :

Ce chapitre met en avant les éléments techniques ayant permis de concrétiser ce projet. L'utilisation de technologies modernes et l'adoption de bonnes pratiques ont été déterminantes pour atteindre les objectifs fixés.

Bibliographie

<https://societelemondeinformatique.com/>

<https://linkweb.fr/creation-site-internet-toulouse/developpement-web/>

<https://www.blogdumoderateur.com/outils-developpeurs-web-debutants/>

<https://getbootstrap.com/docs/5.3/getting-started/introduction/>

<https://fr.wikipedia.org/wiki/Mod%C3%A8le-vue-contr%C3%B4leur>

<https://stackoverflow.com/>

<https://www.tuleap.org/fr/agile/comprendre-methode-agile-scrum-10-minutes#:~:text=Scrum%20est%20une%20des%20m%C3%A9thodes,r%C3%A9guliers%20avec%20les%20utilisateurs%20finaux.>

[20les%20utilisateurs%20finaux.](https://www.tuleap.org/fr/agile/comprendre-methode-agile-scrum-10-minutes#:~:text=Scrum%20est%20une%20des%20m%C3%A9thodes,r%C3%A9guliers%20avec%20les%20utilisateurs%20finaux.)

<https://angular.io>

<https://spring.io/projects/spring-boot>

<https://getbootstrap.com>

<https://keycloak.org>

<https://restfulapi.net>

<https://martinfowler.com>

<https://wikipedia.org>

<https://spring.io/projects/spring-data-jpa>

<https://oracle.com>

<https://w3.org/TR/html5>

<https://typescriptlang.org>

<https://spring.io/projects/spring-cloud-netflix>

<https://spring.io/projects/spring-cloud-config>